

RAG ~ 數位員工

我的開發環境是AMD Ryzen AI Max+ 395 (128GB)的
Windows11上有LM Studio 可以運行Qwen3.6-35B-A3B +
LINUX(VM) 的 Hermes Agent 運行的環境

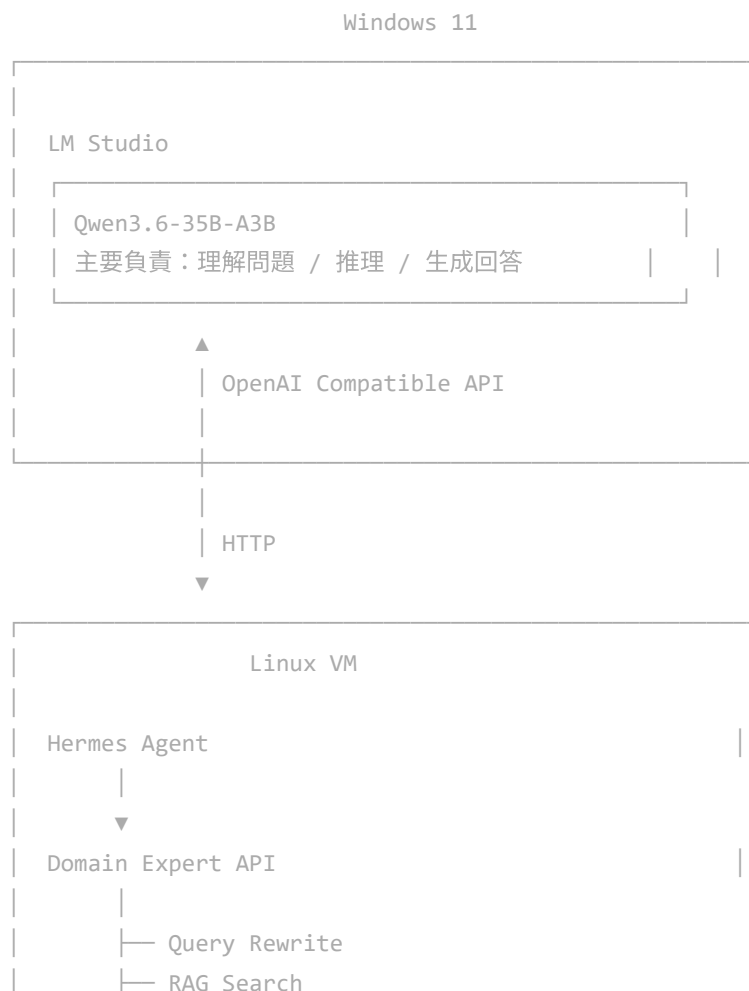
我要如何在此基礎上實作出 一個專屬領域的諮詢專家如 產
品客服/特定領域法律諮詢/...

讓系統只在特定可擴充資料來源的知識庫內進行搜索與回復
現在規劃出在VM內小白也能實作的完整步驟給我

可以，而且你現在這台 **Ryzen AI Max+ 395 / 128GB + Windows 11 + LM Studio + Qwen3.6-35B-A3B + Linux VM + Hermes Agent**，其實很適合做成一個「**RAG → 領域專家 → 數位員工**」的平台。

我會建議你**不要把 Hermes Agent 本身當成 RAG 核心**，而是把它放在「**數位員工 / Agent Orchestrator**」的位置。

整體架構建議做成：





LM Studio 本身已提供 OpenAI-compatible API，包括 `/v1/chat/completions`、`/v1/responses`、`/v1/embeddings`，所以很適合讓 Linux VM 的 RAG 程式直接呼叫 Windows 上的模型。 [LM Studio +1](#)

而 Hermes 本身也支援自訂 OpenAI-compatible endpoint，因此你的架構可以保留 Hermes 作為 Agent 層。 [GitHub +1](#)

一、先定義你真正要做的東西

你不是單純做：

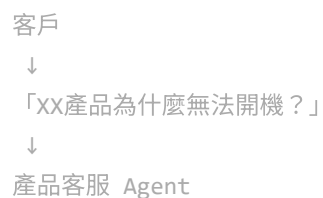
「PDF + ChatGPT」

而是做：

Domain Digital Employee

例如：

產品客服員工



↓
搜尋：
├ 使用手冊
├ 維修手冊
├ FAQ
├ SOP
└ 已知問題
↓
Qwen3.6
↓
「依據產品手冊第 32 頁.....」

法律領域員工

例如：

客戶
↓
「這種租約可以提前解約嗎？」
↓
法律知識 Agent
↓
只搜尋：
├ 法規
├ 法院判決
├ 官方函釋
├ 自建法律資料
└ 內部法律文件
↓
Qwen
↓
引用來源
↓
回答

而且要有一個非常重要的規則：

知識庫沒有證據 → 不回答。

而不是：

「Qwen 大概知道，所以回答看看。」

這是你這個系統與一般 ChatGPT 最大的差異。

二、我建議你的第一版技術選型

不要一開始搞太複雜。

我建議：

元件	技術
OS	Linux VM / Ubuntu
Agent	Hermes Agent
LLM	Windows LM Studio
LLM	Qwen3.6-35B-A3B
Embedding	Qwen3-Embedding-0.6B
Vector DB	Qdrant
RAG API	Python + FastAPI
Document Parser	Python
PDF	PyMuPDF
DOCX	python-docx
TXT/MD	Python
API	FastAPI
Container	Docker Compose
Frontend	第一階段不用
Database	SQLite
Auth	第一階段 API Key
搜尋	Dense → Hybrid → Rerank

Qwen3-Embedding 系列目前有 0.6B / 4B / 8B 等版本；0.6B 支援 100+ 語言、32K context、最高 1024 維 embedding，很適合先拿來做你的中文 RAG。🤗 Hugging Face +1

另外也可以使用 BGE-M3；它是 1024 維、多語言 embedding，而且可以進一步做 dense / sparse / hybrid retrieval。🤗 Hugging Face

你的第一版我推薦 Qwen3-Embedding-0.6B。

三、最重要的觀念：不要把所有文件丟在一起

這一點會直接決定你的系統未來能不能變成商品。

不要：

```
knowledge/  
├─ 所有PDF  
├─ 所有Word  
├─ 所有法律  
├─ 所有產品  
└─ 所有FAQ
```

應該設計成：

```
knowledge/  
├─  
│   └─ product_A/  
│       ├── manuals/  
│       ├── faq/  
│       ├── sop/  
│       └─ warranty/  
│  
│   └─ product_B/  
│       ├── manuals/  
│       ├── faq/  
│       └─ sop/  
│  
│   └─ legal_tw/  
│       ├── laws/  
│       ├── regulations/  
│       ├── court/  
│       └─ interpretations/  
│  
└─ company_internal/  
    ├── hr/  
    ├── finance/  
    └─ sop/
```

然後每一個 chunk 都建立 metadata：

```
{  
  "tenant": "company_a",  
  "domain": "product_support",  
}
```

```
"product": "product_A",
"document_type": "manual",
"version": "2026.09",
"source": "manual.pdf",
"page": 32,
"authority": "official",
"effective_date": "2026-09-01"
}
```

這會讓你未來可以做到：

使用者 A

↓

只能搜尋 company_a

產品客服 Agent

↓

只能搜尋 product_support

Product A

↓

只能搜尋 product_A

這就是「知識隔離」。

Qdrant 本身支援 metadata/payload，也可以進一步做 hybrid search；它可以把 semantic search 與 keyword/lexical search 結合，對產品型號、法條編號、SKU、錯誤碼這種「精確字串」特別有用。  Qdrant +1

四、第一階段：建立 Linux VM

我以下全部以 Ubuntu 為例。

進入 VM：

```
sudo apt update
```

```
Bash
```

安裝基本工具：

```
sudo apt install -y \
git \
curl \
```

```
wget \  
unzip \  
python3 \  
python3-pip \  
python3-venv
```

確認：

Bash

建議：

Python 3.11+

五、建立專案

建立：

```
mkdir -p ~/domain-ai
```

Bash

建立：

```
mkdir -p \  
  app \  
  data/documents \  
  data/processed \  
  data/chroma \  
  config \  
  scripts \  
  logs
```

最後：

```
domain-ai/  
|  
├─ app/  
|  
├─ config/  
|
```

```
├─ data/
│   ├── documents/
│   ├── processed/
│   └─ chroma/
├─ scripts/
└─ logs/
```

六、先安裝 Docker

Bash

啟動：

```
sudo systemctl enable docker
```

Bash

讓目前使用者不用 sudo：

Bash

重新登入 VM。

確認：

```
docker --version
```

Bash

七、建立 Qdrant

建立：

Bash

放入：

```
services:

  qdrant:
    image: qdrant/qdrant:latest
    container_name: domain-qdrant
    restart: unless-stopped

  ports:
    - "6333:6333"
    - "6334:6334"

  volumes:
    - ./data/qdrant:/qdrant/storage
```

啟動：

Bash

確認：

Bash

應該看到：

```
domain-qdrant
```

Qdrant 就負責：

「幫你記住所有知識片段，以及它們的向量。」

八、Windows LM Studio 設定

這個地方非常重要。

你的：

```
Windows
└─ LM Studio
    └─ Qwen3.6-35B-A3B
```

必須讓 Linux VM 可以連進去。

LM Studio 的 Developer 頁面可以啟動本地 API Server，也可以使用 CLI：

```
Bash
```

而 OpenAI-compatible API 預設可以透過 `http://localhost:1234/v1` 使用。  LM Studio +1

但是：

Linux VM 裡面的 localhost ≠ Windows localhost

所以你的 VM 不能直接：

```
http://localhost:1234
```

要使用：

```
http://Windows主機IP:1234
```

例如：

```
http://192.168.1.100:1234/v1
```

九、LM Studio 建議載入兩個模型

Model 1

你的：

```
Qwen3.6-35B-A3B
```

負責：

```
理解
推理
生成回答
```

Model 2

下載：

Qwen/Qwen3-Embedding-0.6B-GGUF

這個負責：

文件 → Vector

問題 → Vector

Qwen 官方 collection 有提供 Qwen3 Embedding 的 GGUF 版本。 🤗 Hugging Face

例如：

Qwen3-Embedding-0.6B

十、測試 Windows → Linux

在 Windows 找到 IP：

PowerShell

例如：

IPv4 Address:

192.168.1.100

Linux VM：

Bash

如果成功，你會看到模型 JSON。

這一步非常重要。

必須先做到：

Linux VM



HTTP



Windows



LM Studio



Qwen3.6

成功之後才繼續。

十一、建立 Python RAG 環境

Bash

建立：

Bash

啟動：

Bash

安裝：

```
pip install \  
fastapi \  
uvicorn \  
openai \  
qdrant-client \  
pydantic \  
python-dotenv \  
pymupdf \  
python-docx \  
python-multipart \  
sentence-transformers
```

十二、建立設定檔

Bash

例如：

```
LMSTUDIO_URL=http://192.168.1.100:1234/v1
```

```
CHAT_MODEL=Qwen3.6-35B-A3B
```

```
EMBED_MODEL=Qwen3-Embedding-0.6B
```

```
QDRANT_URL=http://localhost:6333
```

```
COLLECTION_NAME=domain_knowledge
```

```
TOP_K=8
```

```
MIN_SCORE=0.45
```

注意：

```
192.168.1.100
```

換成你的 Windows IP。

十三、建立知識庫目錄

例如我們先做：

```
data/documents/product_a/
```

放：

```
產品使用手冊.pdf
```

```
產品FAQ.pdf
```

```
產品維修手冊.pdf
```

```
產品保固規則.docx
```

十四、第一個核心：文件切割

這裡是 RAG 最容易被忽略的地方。

不能直接：

```
PDF → 一整份 → embedding
```

應該：

```
PDF
↓
Page
↓
Paragraph
↓
Chunk
↓
Embedding
```

例如：

```
Chunk 001
產品無法開機時，首先確認電源指示燈.....
```

metadata：

```
{
  "product": "product_a",
  "document": "使用手冊.pdf",
  "page": 32,
  "chunk": 1
}
```

十五、建立第一版 ingest.py

建立：

```
Bash
```

第一版可以先做到：

PDF
DOCX
TXT
MD

流程：

文件
↓
讀取
↓
文字清理
↓
Chunk
↓
Embedding
↓
Qdrant

你後面可以再增加：

Excel
PPT
HTML
圖片 OCR
掃描 PDF OCR
Email
網站
API
資料庫

這就是你的「可擴充資料來源」。

十六、Chunk 不要切得太大

第一版建議：

chunk size:
800~1200 tokens

overlap:
100~200 tokens

例如：

原始文件



Chunk 1



overlap



Chunk 2



overlap



Chunk 3



這樣搜尋準確率通常比整份文件直接 embedding 好很多。

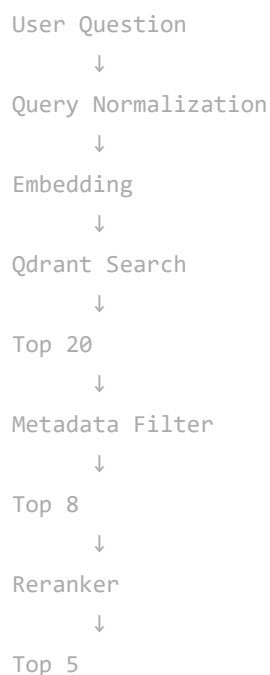
十七、真正的 RAG 流程

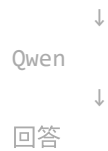
使用者：

「Product A 出現 E102 是什麼意思？」

系統不要直接叫 Qwen 回答。

而是：





十八、我特別建議加入 Reranker

如果你的知識庫只有：

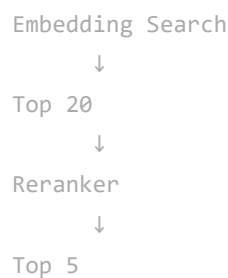
100份文件

可以先不用。

但是如果變成：

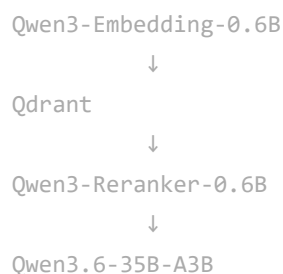
10,000份文件
100,000 chunks

建議：



Qwen3 系列本身也有 Qwen3-Reranker 0.6B / 4B / 8B 。 🤖 Hugging Face

未來可以做：



這會比：

Vector DB → Qwen

更適合正式產品。

十九、最重要：建立「回答防火牆」

這是你的「專屬領域專家」核心。

System Prompt 不要只寫：

你是一個產品客服。

而是：

你是一個受限制的產品知識庫客服 Agent。

你只能根據提供的 KNOWLEDGE CONTEXT 回答。

規則：

1. 不得使用模型自身知識補充答案。
2. 不得猜測。
3. 不得編造文件內容。
4. 如果 KNOWLEDGE CONTEXT 沒有足夠資訊，必須回答：
「目前知識庫沒有足夠資料回答此問題。」
5. 每一個重要結論必須提供來源。
6. 不得引用不存在的來源。
7. 如果不同文件內容衝突，必須指出衝突。
8. 優先使用最新版本文件。
9. 只允許回答指定 domain 的內容。

這才是真正的：

Domain Restricted AI

二十、回答格式也固定

例如要求 Qwen：

```
{
  "answer": "...",
  "confidence": "high",
  "sources": [
    {
      "document": "產品手冊.pdf",
      "page": 32
    }
  ]
}
```

```
  ],  
  "insufficient_information": false  
}
```

LM Studio 的 API 本身支援 structured output / JSON schema，因此你可以要求模型按照固定 JSON 格式輸出。  LM Studio

這非常適合你的系統。

二十一、API 介面

你的 FastAPI 最後提供：

```
POST /api/v1/chat
```

例如：

```
{  
  "tenant": "company_a",  
  "domain": "product_support",  
  "product": "product_a",  
  "question": "E102 是什麼意思？"  
}
```

返回：

```
{  
  "answer": "依據產品使用手冊.....",  
  "sources": [  
    {  
      "document": "產品使用手冊.pdf",  
      "page": 32  
    }  
  ],  
  "knowledge_found": true  
}
```

二十二、這時候才讓 Hermes 介入

你的 Hermes 不應該自己去：

PDF
↓
Vector DB

而應該把 RAG 做成一個 Tool 。

例如：

```
Hermes
|
├─ search_knowledge()
|
├─ get_document()
|
├─ search_product()
|
├─ search_sop()
|
└─ create_ticket()
```

例如 Hermes 收到：

「客戶說 Product A 無法啟動，要怎麼處理？」

Hermes：

```
思考
↓
search_product()
↓
取得知識
↓
search_sop()
↓
取得處理流程
↓
整理
↓
回答客戶
```

這就開始從：

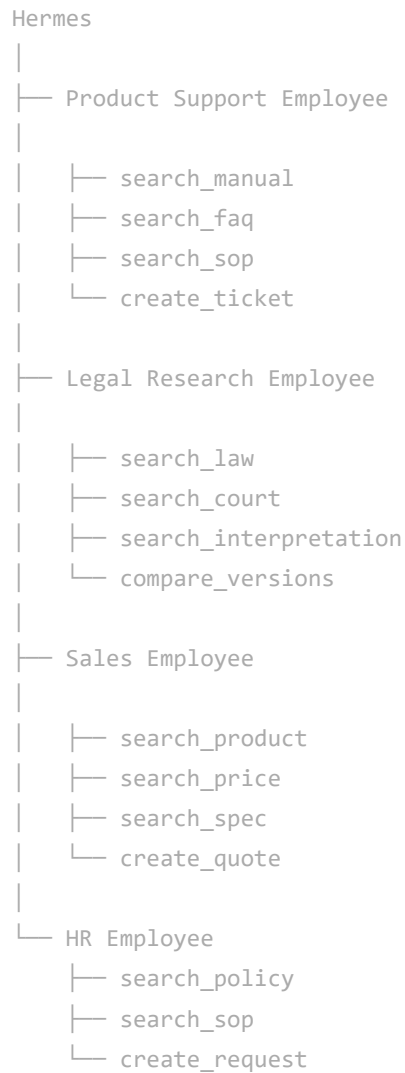
RAG Chatbot

升級成：

AI Digital Employee

二十三、你可以做成「技能型數位員工」

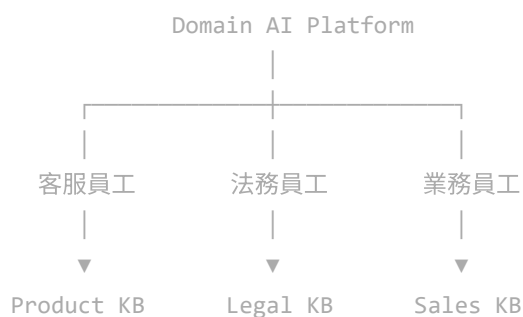
例如：



這就是你真正應該發展的方向。

二十四、甚至可以讓一套 RAG 支援很多「員工」

例如：



而底層：

Qwen3.6
Qdrant
FastAPI
Hermes

全部共用。

二十五、我建議你一定要做「Knowledge Namespace」

例如 Qdrant metadata：

```
{  
  "tenant_id": "company_a",  
  "domain": "product_support",  
  "knowledge_base": "product_a",  
  "document_type": "manual",  
  "version": "2026.09",  
  "language": "zh-TW",  
  "source": "manual.pdf",  
  "page": 32  
}
```

搜尋：

```
tenant_id = company_a  
AND  
domain = product_support  
AND  
knowledge_base = product_a
```

這樣：

Product A 客服

看不到：

Product B
法律
HR
財務

這對未來 SaaS 化非常重要。

二十六、知識庫新增文件的流程

未來你只需要：

```
data/documents/product_a/
```

放：

```
new_manual.pdf
```

然後：

```
Bash
```

系統：

```
new_manual.pdf
```

↓

```
解析
```

↓

```
Chunk
```

↓

```
Embedding
```

↓

```
Qdrant
```

↓

```
完成
```

不用重新訓練 Qwen。

這就是 RAG 最大的價值之一。

二十七、甚至可以做「自動監控」

正式版本可以：

```
data/documents/
```

↓

```
File Watcher
```

↓

```
發現新文件
```

↓

```
自動解析
```

↓

自動 Vectorize



Qdrant

因此：

業務人員



丟 PDF



系統自動更新知識

完全不需要工程師介入。

這就是很典型的數位員工架構。

二十八、法律 AI 要特別設計

如果你之後做「特定領域法律諮詢」，我建議不要做成：

「AI 律師」

而是：

法律資料檢索與分析助手

因為法律資料有：

法規版本

生效日期

修法日期

法院判決

函釋

主管機關

地域

案件類型

所以 metadata 要更完整：

```
{
  "jurisdiction": "TW",
  "law": "民法",
  "article": "第450條",
  "effective_from": "2026-01-01",
  "effective_to": null,
```

```
"source_authority": "官方",  
"document_type": "law"  
}
```

使用者問：

2024 年簽的合約適用哪個版本？

搜尋就不能只靠 semantic similarity。

必須：

```
語意搜尋  
+  
日期 Filter  
+  
法規版本 Filter  
+  
法域 Filter
```

所以你前面做的 metadata 架構會非常重要。

二十九、你的「可信回答」應該有三層

我非常建議你做：

Level 1

```
Knowledge Found
```

找到可靠資料。

Level 2

```
Knowledge Conflict
```

不同資料互相矛盾。

Level 3

```
Knowledge Missing
```

資料庫沒有資料。

例如：

✖ 不要：

「我認為應該是.....」

而是：

⚠ 知識庫目前沒有足夠資料回答此問題。

目前搜尋到：

A文件

B文件

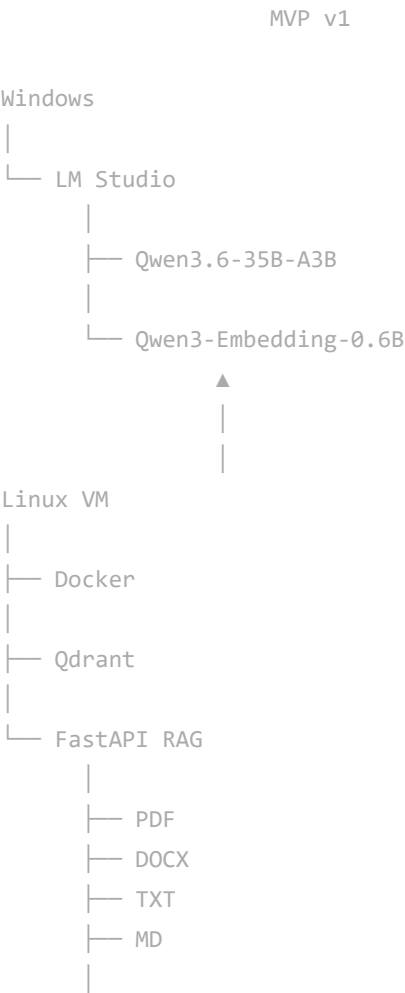
但兩者沒有提供足以確認答案的資訊。

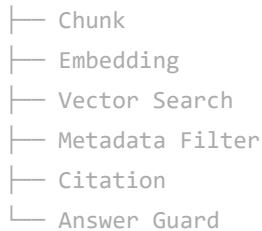
這會大幅降低幻覺。

三十、你的第一個 MVP，我建議只做到這裡

不要一開始就做 30 個功能。

先完成：





先不要做 Web UI。

直接用：

Bash

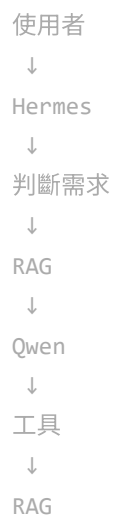
測試。

三十一、MVP 完成後再加 Hermes

第二階段：



變成：



↓
最終答案

三十二、第三階段才做 Web

到這時候才加入：

```
graph TD
    Browser --> FastAPI
    FastAPI --> Auth
    Auth --> DomainRouter[Domain Router]
    DomainRouter --> Hermes
    Hermes --> RAG
    RAG --> Qwen
```

Frontend 可以選：

```
React
Vue
Svelte
```

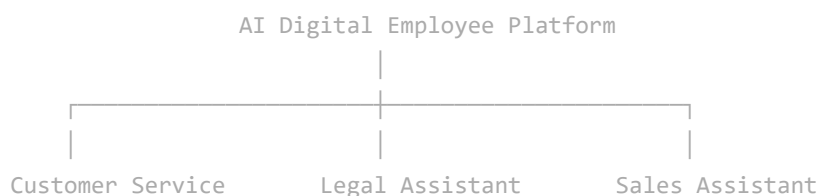
但如果你是自己開發，我反而建議第一版：

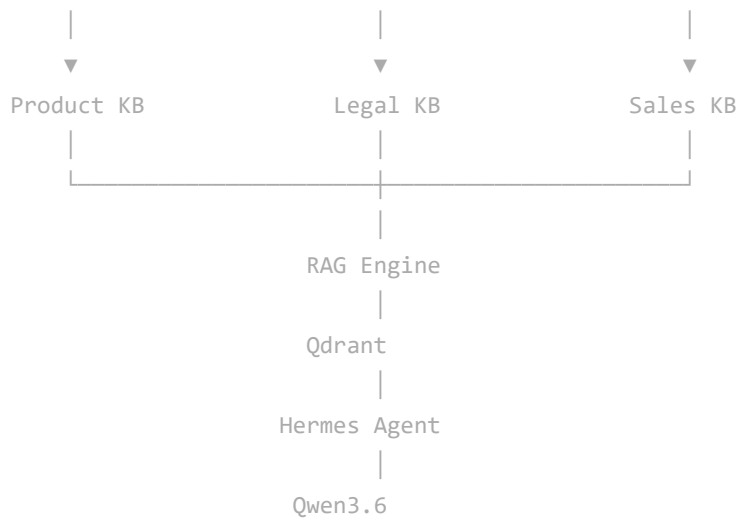
```
FastAPI
+
簡單 HTML/JS
```

先不要浪費時間做漂亮 UI。

三十三、第四階段：變成真正產品

最後可以變成：





然後提供：

新增員工
 新增知識庫
 上傳文件
 設定權限
 設定回答規則
 設定工具
 查看對話
 查看引用來源
 查看錯誤

三十四、最後會形成這個產品模型

你未來甚至可以讓客戶自己：



客戶完全不需要懂：

Vector
Embedding
RAG
Qdrant
LLM
Prompt
Agent

他只知道：

「我上傳公司的文件，就得到一個懂我公司資料的 AI 員工。」

這才是很有商業價值的產品。

三十五、你的環境，我會這樣分工

元件	負責
Ryzen AI Max+ 395 / 128GB	整個平台運算
Windows 11	Host
LM Studio	LLM Server
Qwen3.6-35B-A3B	推理 / 回答
Qwen3-Embedding-0.6B	Embedding
Linux VM	AI Backend
Docker	基礎服務
Qdrant	Vector Database
FastAPI	RAG API
Hermes	Agent / Digital Employee
PDF/DOCX parser	知識匯入
Metadata	知識隔離
Reranker	搜尋精度

元件

負責

Web UI

最後才做

LM Studio 已經提供 OpenAI-compatible API 與 tool-use 能力，所以你現在的 Qwen + LM Studio 不需要重新換掉；你的主要工作是把 **RAG Engine** 與 **Hermes Agent** 層建立起來。

◦  LM Studio +1

我建議我們下一步直接「實作」，不要再停留在架構

我可以按照你現在的 **Windows 11 + Linux VM** 環境，直接帶你從 **0** 開始做一個可以真的跑的 **domain-ai** 專案，順序如下：

```
STEP 01  Linux VM 環境檢查
STEP 02  Docker 安裝
STEP 03  Qdrant 啟動
STEP 04  Windows ↔ VM ↔ LM Studio 網路打通
STEP 05  LM Studio Qwen3.6 API 測試
STEP 06  Qwen3-Embedding 安裝
STEP 07  建立 FastAPI RAG Server
STEP 08  PDF / DOCX / TXT 匯入
STEP 09  Chunk
STEP 10  Embedding
STEP 11  Qdrant 建庫
STEP 12  Metadata / Namespace
STEP 13  RAG Search
STEP 14  Reranker
STEP 15  Citation
STEP 16  「沒有資料就拒答」機制
STEP 17  Hermes Tool
STEP 18  Hermes Domain Expert
STEP 19  API Key / 使用者權限
STEP 20  Web UI
STEP 21  多知識庫
STEP 22  多 AI 員工
STEP 23  Docker 一鍵啟動
```

而且我建議**第一個實例**直接做「**產品客服數位員工**」，不要先做法律，因為可以用你自己的產品手冊/FAQ 做測試，不會一開始就碰到法律版本、法域與高風險回答等複雜問題。

如果照這條路做完，你最後會得到一個真正可以執行的：

```
domain-ai/
├─ docker-compose.yml
├─ .env
├─ app/
```

```
|   ├── main.py  
|   ├── rag.py  
|   ├── qdrant.py  
|   ├── embeddings.py  
|   ├── llm.py  
|   ├── security.py  
|   └── prompts/  
  
└── ingestion/  
    ├── pdf.py  
    ├── docx.py  
    ├── chunker.py  
    └── ingest.py  
  
└── hermes/  
    └── skills/  
  
└── data/  
    └── documents/  
  
└── scripts/  
    ├── setup.sh  
    └── test.sh
```

下一步最適合直接從 STEP 01 開始，我可以直接把「Ubuntu VM → Docker → Qdrant → LM Studio 網路 → Python RAG」所需要的所有指令與完整檔案內容一次給你，做到最後 curl 問一句話就能得到「答案 + 文件來源 + 頁碼」。



CambridgeNexus, Inc. 廣告 ...
Make GPU Capacity Predictable
Reserve dedicated NVIDIA GB300 capacity
for your training and production roadmap.

廣告不會影響你從 ChatGPT 取得的回答。你的聊天內容將保持私密。了解廣告和個人化設定 >

ChatGPT 可能會出錯。請查核重要資訊。